



LOVELY
PROFESSIONAL
UNIVERSITY

Transforming Education Transforming India

Capstone report

A Comprehensive Review of Robust and Explainable
Deep Learning Frameworks for Stock Strategy
Backtesting

Submitted by:

Anuska Palit

Priyanshu Priyadarshi

G Raviteja Reddy

Harshini Gangadhara

Sushil

Ayush Kharkwal

Submitted to:

Faculty: Anurag Tewari

1. Introduction

Recent advancements in machine learning, high-frequency data infrastructure, and deep neural architectures have heavily impacted the development of modern financial technology systems, yet stock strategy backtesting—the primary means by which we evaluate quantitative trading models—remains seriously limited by unrealistic assumptions, poor modeling of intricate market behaviors, and an inability to interpret. For example, backtesting engines mandate the use of colorful historical simulations, which rest on the assumptions of perfect liquidity, instantaneous execution, stationary price behavior, and no cross-asset dependencies. It goes without saying, these assumptions are very much at odds with the real-world structure of how financial markets function. In the financial markets, slippage, latency, dynamic liquidity, volatility clustering, and systemic contagion drive the efficacy of several strategy implementations.

In the past few years, deep learning-based frameworks have generated credible ways of improving the realism and dependability of stock strategy backtesting environments. Temporal Graph Neural Networks (TGNNs) provide the ability to model financial contagion and cross-asset dependencies; adversarial and generative learning frameworks can mimic extremely low-probability market shocks that are beyond the training historical sample; reinforcement learning allows sequential decisions under uncertainty; and explainable AI (XAI) approaches contribute to model behavior and interpretation under complex market events. While involved research remains varied (including forecasting, execution-aware simulation, stress testing, and interpretability) these developments are still done in silos, resulting in commoditized backtesting systems that often remain incomplete and inadequate for utilization in live trading.

So the current project, "A Deep Learning-Based Framework for Automated Backtesting and Strategy Optimization", aims to address these fundamental challenges through the development of a modular, integrated and execution-aware backtesting system. More specifically we propose to integrate guidelines for the construction of an execution-aware backtesting system that incorporates TGNN based contagion modeling, using generative exploration for adversarial stress-testing scenario generation, reinforcement-learning based optimization and counterfactual explainability for traders and institutions to utilize next generation frameworks for evaluation of trading strategies. More broadly, automated multi-objective optimization (as decision aids), better transaction cost modeling, and live walk-forward validation pipelines will also be integrated in our exploration of a fully automatable evaluation system. Collectively, these enhancements will help mitigate manual evaluations of trading strategies from weeks to minutes. This unified proposal will strive to bridge new research initiatives with practical applications in order to improve decision-making, more effective and credible/robust outcomes, and enhance responsible experimentation of financial capacities.

1.1 Objective of the Project

The main goal of this project is to develop a deep-learning-based automated backtesting engine that will evaluate trading strategies with more accuracy, robustness, and interpretability than historically simulated systems. The specific objectives are:

- i. Build an end-to-end automated backtesting engine which supports the near real-time evaluation of trading strategies under both historical and synthetic market scenarios.
- ii. Integrate execution-aware components such as slippage modeling, bid–ask spreads, order-book flows, and transaction cost estimation to provide a more realistic market execution experience.
- iii. Model cross-asset relationships and contagion through a Temporal Graph Neural Network (TGNN) so that the backtesting engine can add spillovers, co-movement, and dynamic asset dependencies to the algorithms being reviewed.
- iv. Utilize generative adversarial models and diffusion-based stress tests to expose strategies to extreme, rare, and adversarial market conditions.
- v. Implement reinforcement-learning-based optimization procedures to tune strategy parameters, position sizing, and execution decisions under uncertainty.
- vi. Provide explainable AI tools such as attribution analysis and counterfactual reasoning to increase transparency and trust in algorithmic decisions.
- vii. Design an interactive dashboard to provide structured, interpretable, and user-friendly visualizations of performance metrics, stress-testing results, and model-based explanations.
- viii. Support scalability across asset classes including equities, forex, crypto, and ETFs.

Ultimately, the goal is to build a thorough, multi-layered ecosystem that advances backtesting from a simplistic historical replay to a realistically simulated, adversarially tested, and transparently explained assessment environment that is relevant in academic research and industry practice.

1.2 Description of the Project

This project proposes the creation of a comprehensive deep-learning-oblivious backtesting and strategy optimization framework that integrates multiple elements of advanced sophistication into a consistent system. The proposed system will utilize multi-modal modeling technologies that are both dynamic and data-based, in contrast to traditional backtesting engines that utilize fixed price histories and deterministic simulations. The multi-layer architecture identified in the system will consist of:

1. Data Layer

- Converts price signals into multivariate time series, graph-structured matrices, or tensor formats.
- Ensures feature engineering pipelines (volatility, momentum, liquidity indicators, spreads).

2. Execution-Aware Simulation Layer

- Models slippage, latency, spread costs, & depth-based fill probabilities.
- Uses neural execution models to approximate microstructural behavior under varying conditions.

3. TGNN-Based Contagion Modeling Layer

- Each asset is represented as a node in a dynamic graph.
- Learns temporal dependencies using attention-based graph architectures.
- Models co-movement during crises, sector-level interactions, & spillover volatility.

4. Generative / Adversarial Stress Testing Layer

- GANs, diffusion models, & adversarial perturbations generate market sequences outside of the historical sample.
- Simulates extreme crashes, liquidity droughts, correlated shocks, & non-linear behaviors.

5. Reinforcement Learning Optimization Module

- Implements strategy optimization using DQN, PPO, SAC or distributional RL.
- Supports multi-objective reward functions balancing returns, risk, & stability.

6. Explainable AI, & Counterfactual Analysis

- Provides model interpretability using SHAP, integrated gradients, & causal/counterfactual frameworks.
- Explains how volatility, spreads, or contagion led to strategy outcomes.

7. Visualization & Reporting Dashboard

- Results presented with equity curves, risk metrics, stress-test outputs, & explanations.
- Increases user interactivity via drill-down analytics, & scenario-based evaluation.

The system architecture allows modularity, scalability, and flexibility, therefore, allowing for the addition of new models, different datasets, and the ongoing evolution of methods of research. The synergy enables an overall cost-effective and computationally intensive environment that meets regulatory standards, research rigor and is applicable to the industry.

1.3 Scope of the Project

The project's scope encompasses the full range of backtesting, stress-testing, strategy optimization, and model interpretability. As financial market data is multidimensional and modern machine learning algorithms are complex, the project's scope is determined across these dimensions:

1.3.1 Technical Scope

- The establishment of backtesting modules embedded with deep learning.
- Modeling of market contagion through graph neural networks.
- Simulation with execution awareness using microstructure-based features.
- Adversarial/generative scenario creation.
- Optimization via reinforcement learning.

- Explanatory modules for interpretability and decision-making transparency.
- A dashboard interface for visualization and result interpretation.

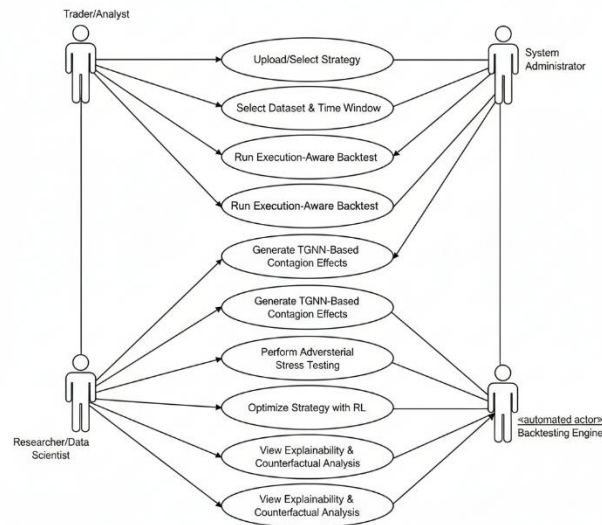
1.3.2 Data Scope:

- Financial historical datasets on a daily and tick level (NYSE, NASDAQ, S&P500, LOBSTER).
- Multi-asset synchronization for cross-asset learning.
- Synthetic datasets from generative models for stress tests to models.
- Metadata for logging, monitoring, evaluation, and tracking pipelines.

1.3.3 Application Scope:

- Quantitative trading system evaluation
- Academic research in financial machine learning.
- Risk management and scenario analysis.
- Strategy optimization for portfolio managers.
- Educational tools for learning financial contagion and deep learning modeling.

1.3.1 Use Case Model



This use-case model defines the primary interactions and functional boundaries of the proposed system.

2. System Description

The system integrates multiple subsystems: a data ingestion and processing layer, an execution-aware simulation engine, a TGNN-based contagion modeling module, adversarial scenario generators, multi-objective optimization algorithms, and an explainable AI interface. Together,

these components form a cohesive and modular pipeline capable of delivering realistic, robust, and interpretable backtesting outcomes suitable for both practitioners and researchers.

2.1 Customer/User Profile

The system is for many types of users with varying degrees of familiarity with a invented financial technology, from various disparate backgrounds in finance, academic, and technical fields. Each user group will interface to the system in its own way and have varying aspects of depth and expertise related its use. Below are the user profiles.

A. Quantitative Traders / Portfolio Managers

The system is generally used to efficiently measure strategies, stress test it to realistic scenarios, and optimize, while minimizing manual interaction with the system.

Primary needs:

- Reliable and realistic execution aware backtests
- Stress testing scenarios for rare or adversarial scenarios
- Risk adjusted metrics (Sharpe, Sortino, CVaR)
- Multi-objective optimization tools
- Transparency into failures of the strategies
- Run backtests, consult the metrics on dashboard, documented the robustness of the strategies.

B. Financial Researchers / Academics

Study the behavior of market dynamics, cross-asset dependencies, deep learning frameworks, and explore stress modeling.

Primary Needs:

- Access to TGNN and future-trained deep-learning models
- A safe space to test hypotheses about contagion, volatility spillovers, and regime shifts
- Detailed log files for reproducibility
- Algorithmic modularity to develop new models

C. Data Scientists / ML Practitioners

Develop, integrate, and test deep learning models within the backtesting framework.

Primary Needs:

- Easy access to clean and standardized time-series data
- A framework to plug in new models (GANs, diffusion models, RL agents, etc.)
- Hyperparameter tuning capabilities

- Debugging and interpretability tool.

As a group, these profiles will help define the limits to usability of the system and form the rationale for a modular, scalable, and interpretable system to support advanced research and practical trading analysis.

2.2 Functional Requirements

The functional requirements define the core capabilities that the system **must** perform. They are grouped into logical modules for clarity.

A. Data Management and Preprocessing

FR-1: The system needs to allow ingestion of historical and synthetic market data sets.

FR-2: The system should preprocess data into suitable formats (time-series tensors, graphs).

FR-3: The system should allow for the preprocessing of common, systematic, derived, or engineered features (volatility, spreads, liquidity, momentum).

FR-4: The system should support synchronizing multi-asset datasets into a common timeframe.

B. Execution-aware backtesting engine

FR-5: The system should simulate the execution of a trade in as realistic a way as possible in a simulated market.

FR-6: The system needs to compute slippage, spread costs, and order fill probabilities.

FR-7: The system needs to provide performance metrics (returns, maximum drawdown, Sharpe ratio, CVaR).

FR-8: The system must support walk-forward and rolling-window validation methods.

C. TGNN-based contagion and multi-asset modeling

FR-9: The system needs to provide an automatic way to create dynamic graph representations of assets.

FR-10: The system needs to run the TGNN models to learn asynchronous and cross-asset dependencies.

FR-11: The system must estimate contagion effects and co-movement during backtesting.

FR-12: The system should allow visual representations of graph embeddings and contagion paths.

D. Dashboard and visualization

FR-22: The system has to visualize performance reports, graphs, and performance metrics.

FR-23: The system has to incorporate tools for comparing scenarios (historical and adversarial).

FR-24: The system has to support exporting of the results, logs, and visualizations.

E. System Management

FR-25: The system is required to maintain experiment logs, audit trails, etc.

FR-26: The system is required to support model checkpointing and version control.

FR-27: The system is required to support access by multiple users with role separation.

2.3 Non-Functional Requirements

Non-functional requirements provide constraints on system performance, quality characteristics, and conditions of use that must also be met to guarantee reliability and scalability.

A. Performance Requirements

NFR-1: Backtesting simulations must be completed much faster than manual methods (minutes rather than weeks).

NFR-2: The system will use GPU-based acceleration for the deep learning modules.

NFR-3: Ingestion of data must scale to include millions of time-series rows without failure.

NFR-4: TGNN inference must be completed with acceptable latency for multi-asset graphs.

B. Scalability Requirements

NFR-5: The system must scale to multiple asset classes (equities, forex, crypto).

NFR-6: The architecture must support easy integration of new models (e.g., new GAN variants).

NFR-7: The dashboard must support large datasets when queried by multiple users.

C. Reliability Requirements

NFR-8: The system must provide the same results when provided with the same input datasets and configurations.

NFR-9: Experiment logs must be maintained across sessions to allow for reproducibility.

NFR-10: The error handling of the system must not allow the system to crash due to data inconsistencies.

D. Security Requirements

NFR-11: User information must have role-based access control.

NFR-12: Data classified as sensitive must have the potential for secure storage and, when appropriate, encryption.

NFR-13: Validation must occur on input to all system modules to minimize code injection.

E. Usability Requirements

NFR-14: The information dashboard must have engaging (interactive) visualizations that are easy to understand and report informative data.

NFR-15: Users should be able to run backtests without having to implement extensive coding.

NFR-16: Documentation and tool tips must be available to assist users with configuring a strategy.

F. Maintainability Requirements

NFR-17: The code base must be modular, lending itself to updating, changing, and general system maintainability.

NFR-18: Configurations of the modules must be stored in a 'user-accessible' format such as YAML or JSON.

NFR-19: Automated tests must be available to run before production deployment to avoid regression.

G. Explainability & Transparency Requirements

NFR-20: The results of all strategies must always contain an interpretable explanation.

NFR-21: Any counterfactuals must illustrate or highlight tangible “what if” scenarios..

3. Design

3.1 E-R Diagram

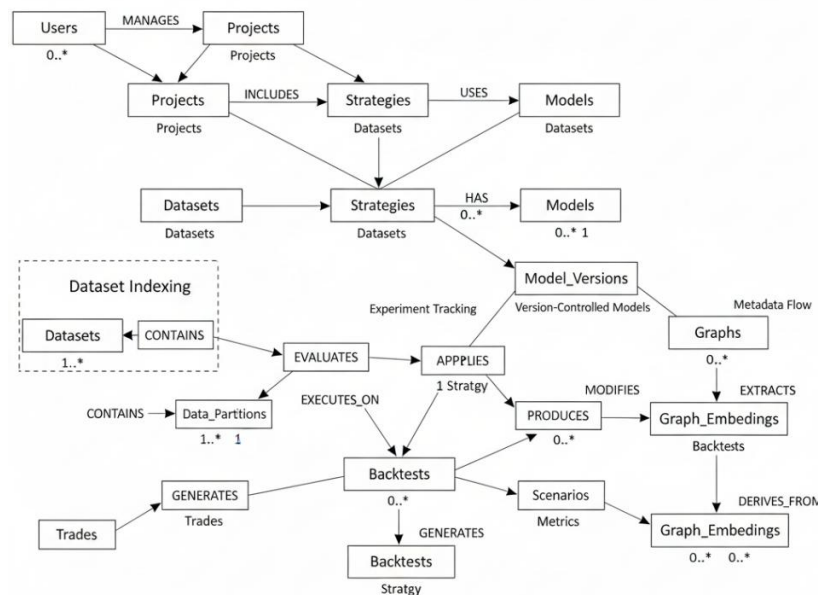


Fig i: ER Diagram for the system

3.2 DFD

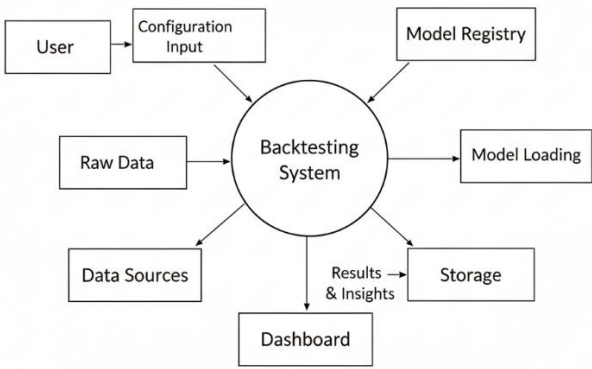


Fig ii: Ai based Automate backtesting

3.3 High-Level System Architecture

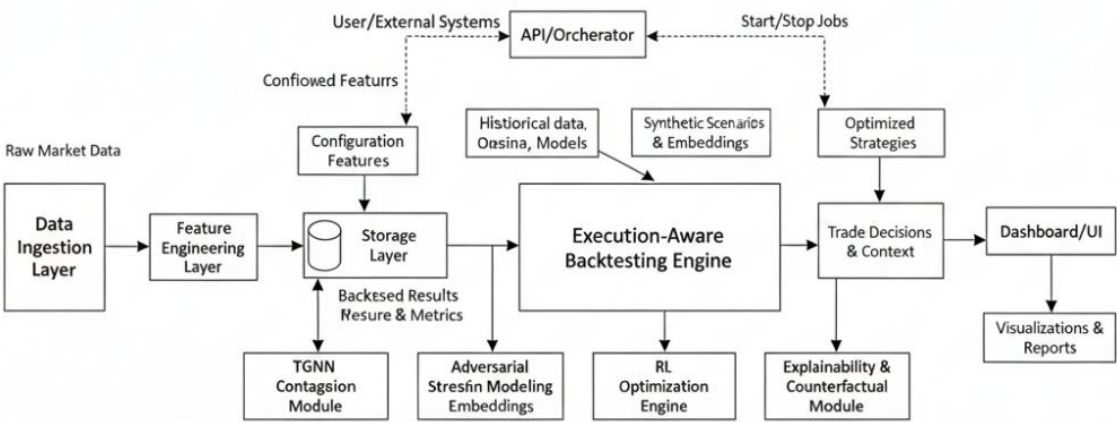
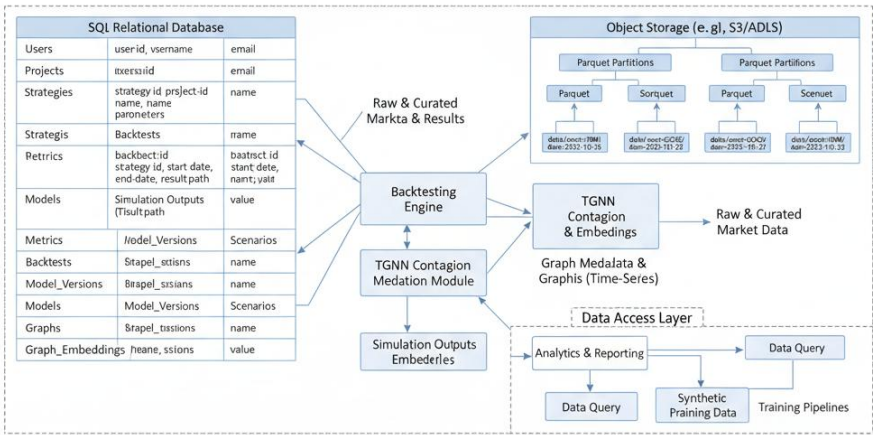
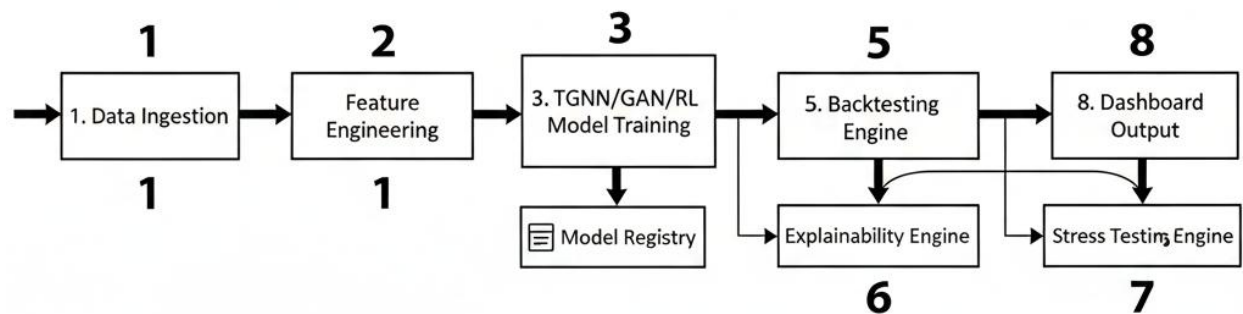


Fig iii: Architecture diagram

3.4 Database Architecture / Storage Design Diagram



3.5 Workflow Diagram (End-to-End System Pipeline)



4. Scheduling and Estimates

The multidisciplinary approach of the project—leave deep learning, financial modeling, graph networks, reinforcement learning, and explainability—requires a structured and incremental development plan. The timeline below is a 24-week (approx. 6-month) project, which is a reasonable timeframe for a research-grade project whose components will use state-of-the-art AI technology.

4.1 Phases of the Project and Milestones

The project will proceed through six (6) core phases with deliverables for each phase:

Phase 1: Requirements Analysis & Literature Synthesis (Weeks 1–3)

- Review academic literature in the following areas: TGNNs, adversarial learning, executing-aware backtesting, and RL.
- Consolidate the problem definition, data set requirements and model selection.
- Refine the system architecture, and scope of the project.
- Prepare early design diagrams (DFD, ERD, and architecture).

Deliverables:

- Finalized requirements document
- Synthesis of research report
- Initial design diagrams

Phase 2: Data Collection & Feature Construction (Weeks 4–7)

- To collect data sets: S&P500/NASDAQ minute data, LOBSTER microstructure data, and Kaggle and tick-level data
- Develop a systematic single feature pipeline.
- Build multi-asset synced matrices and the graph adjacency matrices (correlation, Granger causality).
- Create a Parquet-based feature store.

Deliverables:

- Dataset repository
- Scripts for feature construction.
- Graph construction pipeline

Phase 3: Development of the Core Backtesting Engine (Weeks 8–12)

- Implement execution-aware simulation (slippage, spread, depth-based fills).
- Integrate transaction cost modeling and latency modeling.
- Add the option for walk-forward and rolling-window validation.
- Develop modular strategy API.

Deliverables:

- A backtesting engine that is functional
- A simulation module for executing the plan
- A component that calculates risk metrics

Phase 4: Modules with Deep Learning (Weeks 13-18)

Sub-module A: Modeling contagion with TGNN (Weeks 13-15)

- Train TGNN models on graphs of multiple assets.
- Add graph embeddings to the back-test module.

Sub-module B: Adversarial & Generative Scenario Stress Testing (Weeks 15-17)

- Implement stress-testing generators based on GAN and diffusion.
- Create synthetic sequences and integrate them into the backtest module.

Sub-module C: Optimizer with Reinforcement Learning (Weeks 17-18)

- Implement a wrapper around the RL agent/ environment.
- Train PPO/SAC agents to optimize the strategies.

Deliverables:

- A trained TGNN model
- An experimental scenario generator
- The pipeline used for RL training

Phase 5: Explainability & Dashboard Development (Weeks 19-21)

- Implement SHAP, Integrated Gradients, and compare models with counter-factual logic.
- Create an intuitive dashboard to visualize metrics.
- Add a scenario comparison interface (historical vs relative) to compare models.

- Create experiment logs, audit trail pages.

Deliverables:

- A fully functional XAI module
- A dashboard including visualization suite
- An engine for counter-factual analysis

Phase 6: Evaluation, Testing & Reporting (Weeks 22-24)

- Run full-scale evaluations across datasets.
- Testing for robustness, stress-testing and interpretability.
- Validate reproducibility and runtime performance.
- Create final report and documentation.

Deliverables:

- Complete report an evaluation
- Final documentation of project
- Toolkit ready for deployment

5. Challenges and Solutions

Category	Challenge	Description	Solutions / Mitigation Strategies
Data-Related Challenges	Non-Stationary and Noisy Financial Data	Market regimes change frequently due to volatility spikes, structural breaks, liquidity shifts, and macroeconomic events, making model training unstable.	• Use adaptive learning (sliding windows, regime segmentation, online updates) • Use TGNNs for dynamic dependency modeling instead of static correlations • Apply adversarial augmentation to expose models to extreme volatility
	Synchronizing Multi-Asset and Multi-Frequency Data	Multi-asset datasets are asynchronous, incomplete, and irregularly sampled, making synchronization difficult.	• Apply forward-fill/backfill with confidence weighting • Normalize timestamps across all assets • Build multi-resolution feature pyramids (daily + minute + tick)
Modeling Challenges	TGNNs Require Heavy Computation and Memory	Temporal graph models require continuous graph updates and high-dimensional tensor operations, causing GPU memory strain.	• Use sparse adjacency matrices • Apply minibatch/neighbor sampling (GraphSAGE-style) • Precompute static graphs for low-volatility zones
	Unrealistic Stress Scenarios from GANs/Diffusion Models	Generative models may hallucinate invalid or inconsistent market paths without proper constraints.	• Condition models on volatility regimes, liquidity states, and TGNN embeddings • Enforce temporal consistency (Markov penalties, spectral constraints) • Train discriminators on cross-asset joint distributions
	RL Agents Overfit Historical Patterns	RL policies memorize historical trajectories and fail under new or unseen market regimes.	• Introduce adversarial environment randomization • Apply reward shaping to penalize excessive turnover or variance

Category	Challenge	Description	Solutions / Mitigation Strategies
			• Use distributional RL to account for uncertainty
System & Integration Challenges	Execution Simulation Requires LOB Data	Limit-order-book data is expensive, incomplete, or unavailable for many assets, limiting realistic execution modeling.	• Train neural execution surrogate models to approximate LOB behavior • Use hybrid execution models that fall back to parametric slippage estimators when LOB is missing
	Complex Explainability for Multi-Asset Deep Models	XAI tools designed for tabular or image data cannot directly capture cross-asset temporal relationships.	• Combine TGNN attention maps with strategy-level attributions • Use counterfactual simulations to analyze sensitivity to slippage/spread
	Reproducibility Across Deep Learning Runs	Random seeds, GPU kernels, and parallel processes cause slight differences between runs, reducing reproducibility.	• Enforce deterministic seeds wherever possible Log model versions, configuration files, and hyperparameters • Use hashed dataset partitions and immutable experiment logs

6. Conclusion and Future Directions

6.1 Conclusion

This undertaking tackles a critical gap in quantitative finance: Another integrated, realistic, and explainable backtest environment that can capture the revelatory complexity of modern financial markets. Existing backtest engines rely on overly simplistic assumptions that break down in real-world situations, especially when confronted with slippage, lack of liquidity, volatility shocks, and cross-asset contagion. By leveraging execution-aware simulations, Temporal Graph Neural networks, adversarial and diffusion-based stress generators, reinforcement learning-driven optimization, and counterfactual explainability techniques, the proposed framework provides an avenue for quantitative researchers to go beyond traditional backtests that rely on static, linear replaying of historical price data. Rather, this provides an environment where you can dynamically learn and run in a multi-scenario setting that can adapt to various market regimes. The result offers improved representation of market behavior, increased robustness of trading strategies, and greater confidence by traders and researchers. Collectively, the modular design, comprehensive cataloging of price data sets, model registry infrastructure, and interactive dashboard setup creates an important setting for academic exploration and real-world usage in algorithmic trading environments.

6.2 Future Directions

The system's future possibilities are also very promising. For instance, it would be possible to perform real-time adaptive backtesting when data streaming tools are leveraged, like Kafka or Flume, to ensure the test conditions are consistent with actual market conditions. Another area is multi-agent market simulation, where interacting reinforcement learning agents, such as market makers, retail traders, or institutions, could model realistic market microstructures. We could also

implement causal graph modeling—an intervention-ready analytical strategy—using methods like PCMCI or Neural Granger. We could also apply explainable reinforcement learning methods that would influence policy behavior and allow for understanding how decisions advance across regimes. The system could also be enhanced with regulatory-grade analytics, such as MiFID II, best-execution analytics, or order reconstruction. Further, the system could accommodate other asset classes beyond equities—such as forex, commodities, cryptocurrencies, and derivatives—with asset-class specific execution models. Lastly, LLM's could assist with automated strategy interpretation, error identification, code reviews, and optimization guidance—allowing us to increase knowledge, intelligence, and usability.